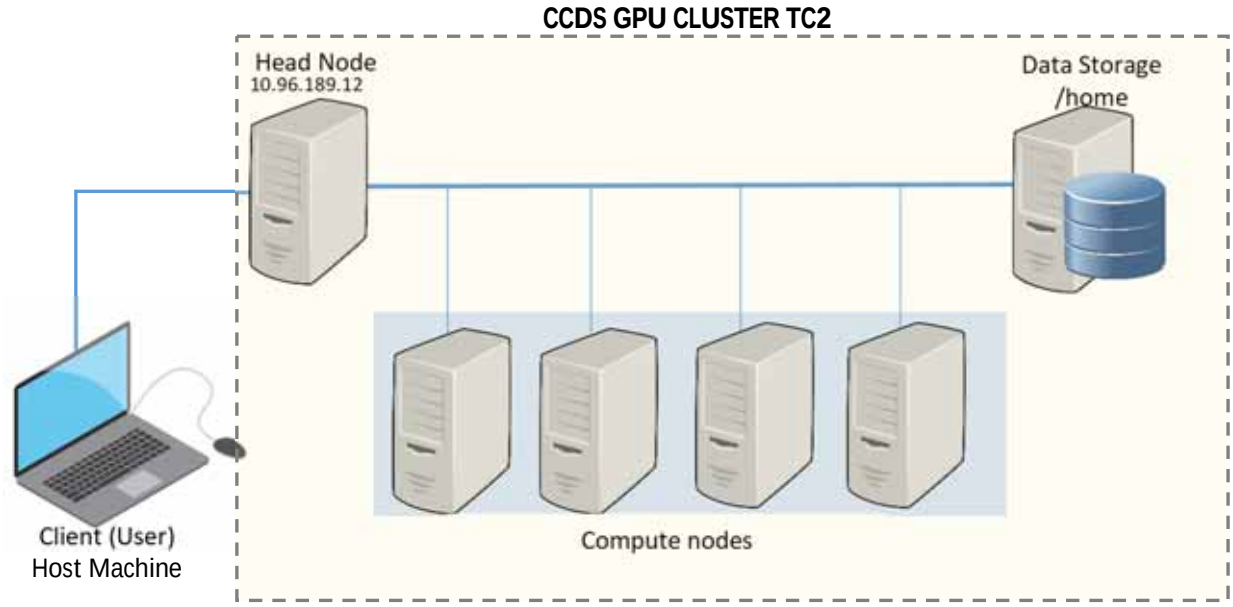
# CCDS GPU CLUSTER TC2 USER GUIDE


| *Content* | *Page* |
|---|---|
| Introduction | 2 |
| Logging Into the Cluster | 4 |
| Conda Package and Environment | 9 |
| SLURM User Guide | 13 |
| Guideline for SLURM Job Submission | 16 |
| Miscellaneous | 22 |
| Other Resources for Application | 26 |
| Important Notice | 27 |

# Introduction




In CCDS GPU Cluster TC2, there are two diverse types of nodes for diverse types of tasks.
The initial node you log in to is called the **Head Node**, serving as the main access point for the cluster.

**The GPU cards for computation are in those Compute Nodes, NOT in the Head Node.**
The users are **not allowed** to compile and run code on the Head Node. The users must create the job script to submit the computation request (knowns as **non-interactive job**) to SLURM (**Simple Linux Utility for Resource Management**) job scheduler, for the system to process in the allocated GPU Compute Nodes.

The available resources for the user to utilise for GPU computation is limited by the assigned **QoS** (SLURM-Quality of Service).

**Client Tools for accessing the CCDS GPU Cluster Head Node**

| Name of Program | Description & Purpose | Where to download |
|---|---|---|
| **PuTTY** | **[SSH Client for Windows]**<br>**PuTTY** is an open-source software as SSH and Telnet Client for *Windows Platform*.<br><br>**SSH** (Secure Socket Shell) is a network protocol that gives users, particularly system administrators, a secure way to access a computer over an unsecured network. | The installer is available online and can be easily located using any search engine.<br><br>**Tip:**<br>Download the portable edition "putty.exe"<br>– no need to install, ready for use<br>*http://www.chiark.greenend.org.uk/~sgtatham/putty/download.html*<br><br>**Other Online References:**<br>*https://www.ssh.com/academy/ssh/putty* |
| | **[ How to SSH on MAC ]**<br><br>**Online References:**<br>*https://www.ssh.com/academy/ssh/putty/mac*<br>*https://www.servermania.com/kb/articles/ssh-mac/* | |

| | | |
|---|---|---|
| **WinSCP** | **[SFTP Client for Windows]**<br>**WinSCP** (Windows Secure Copy) is a free and open-source SFTP, FTP, WebDAV and SCP client for Microsoft Windows<br>**SFTP** (SSH File Transfer Protocol) is an encrypted or secure file transfer protocol. | The official download site for WinSCP:<br>*https://winscp.net/eng/index.php*<br><br>**Installation tip:**<br>Select "*Explorer*" for user interface style<br><br>**Other Online References:**<br>*https://winscp.net/eng/docs/guide_install* |
| **FileZilla** | **[SFTP Client for MAC]** | Download Site for FileZilla:<br>*https://filezilla-project.org/index.php*<br><br>**References for MAC user, apply SFTP commands in the terminal for data transfer:**<br>*https://help.uillinois.edu/TDClient/37/uic/KB/ArticleDet?ID=582*<br>*https://www.servermania.com/kb/articles/ssh-mac*<br>*https://beebom.com/how-to-use-mac-terminal-ftp-sftp-client/*<br>*https://tipsmake.com/use-terminal-on-mac-as-ftp-or-sftp-client* |

## Workflow for the user:

1. Establish your first access via SSH. Your home directory will only be created after your first login

2. Uploading all the necessary job script, coding, dataset from your host machine to the cluster's storage (your home directory) via SFTP client

3. Setup and load your Conda environment(s) with required applications for computation

4. Submitting the non-interactive job using SLURM scheduler

5. Copy the job output back to your host machine, via SFTP client

This guide assumes that you have basic knowledge of accessing the Linux system via command-line interface:

- Connecting to a Linux server via SSH
- Moving files to a Linux server from your local computer, and vice versa
- The Linux basic commands such as: ls, cd, pwd, cp, mv, rm, chmod etc.

**References for Basic Linux Commands:**
*https://serverdale.com/en/linux-commands*
*https://centoshelp.org/resources/commands/linux-system-commands/*
*https://www.guru99.com/must-know-linux-commands.html*

- The GPU cards for your computation are NOT in the Head Node. So, do not attempt to execute command to acquire the GPU information in the Head Node e.g. "*nvidia-smi*", "*lspci | grep -E 'VGA|3D'*" and etc.

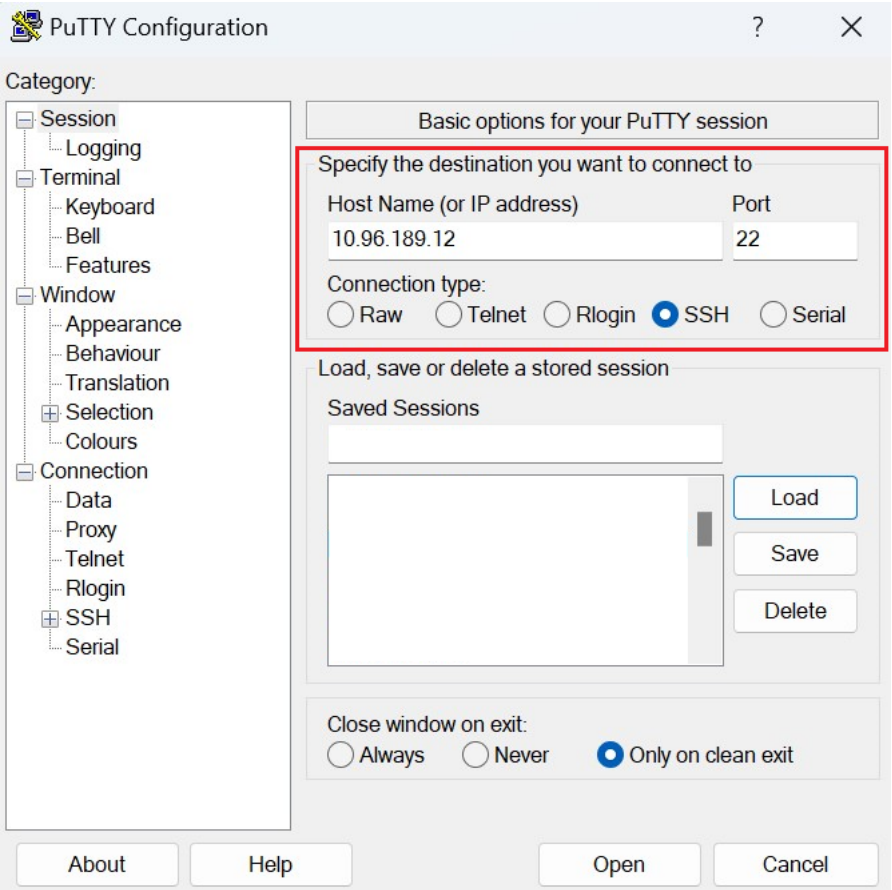
# Logging Into the Cluster

The SCSE GPU Cluster TC2 is accessible **only within NTU network**
For off-campus access, the user must login to NTU Virtual Private Network (VPN) *[ https://ntuvpn.ntu.edu.sg ]*

| | |
|---|---|
| **Cluster** | **CCDS GPU CLUSTER TC2** |
| **IP Address** | **10.96.189.12** |
| **Login Credential** | Your NTU Network Account ID *(in Lower Case)* & Password |

## SSH via PuTTY

1. Launch PuTTY.
   Under the Host Name (or IP address) enter [10.96.189.12].
   Ensure that the Port is 22.




2. Examples of command-line access via your LINUX or MAC Terminal:
   *>> ssh -l <Your Username in Lower Case> 10.96.189.12*
   *>> ssh -p 22 <Your Username in Lower Case>@10.96.189.12*

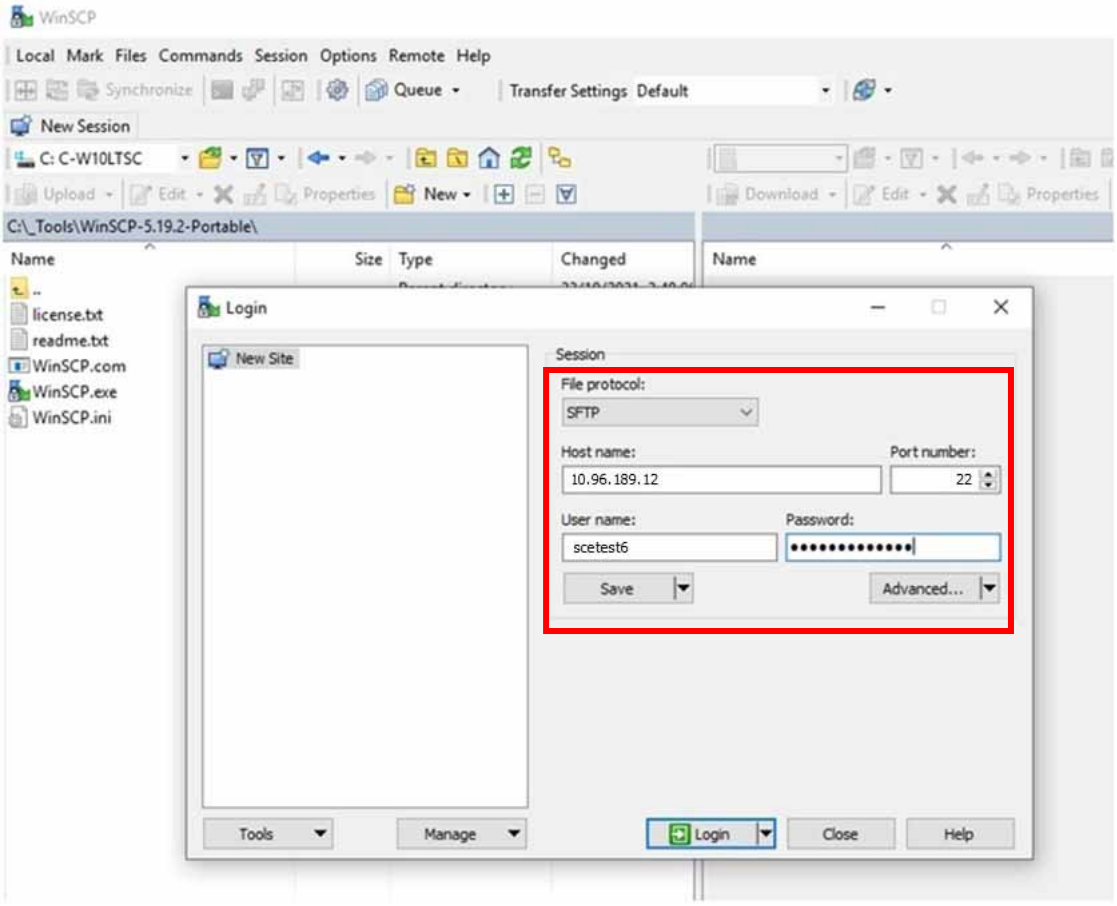
3. On your first login, you will be asked to accept the host key.
   Do click "**yes**" to continue, and you should get a terminal window, where you will be prompted for your credentials. Log in with your NTU Network Account ID (*in Lower Case*) and password.

```
| /-----------------------------------------------------------------------\
|                 **IMPORTANT NOTICE for Users of TC2**
>
>
|
|  1. System Administrators claim the right to remove the accounts of
|     users and the folders assigned to users without prior notice.
|
|  2. NO backups will be provided for data stored in folders assigned to
|     users.  Users are responsible for protecting and maintaining their
|     own data.
|
|  3. Additional user folders and additional QoS assigned to users will
|     be removed after the usage expiry date, without prior notice.
|
|  4. Resources are to be used for academic purposes only. Use of
|     resources for purposes other than that for which they were granted
|     will result in the termination of the account.
|
|  5. Your computation is being monitored. For jobs submitted with
|     unauthorised QoS (NOT assigned for your use) and operations, will
|     be terminated with no prior notification. Severe offenders may be
|     barred from the service.
|
|  6. DO NOT access directories other than your Home Directory or
|     Shared Folder. Unauthorised access to other directories or files,
|     even for the purpose of "browsing", shall be deemed as a security
|     breach.  Serious action will be taken against users committing
|     any breach to the CCDS GPU Cluster.
|
|  7. The user guide (PDF format) for this GPU Cluster is available
|     for download from the share folder: /tc2share.
|
|  8. You may execute the command "mytcinfo" to view your user
|     information.
|
|  9. For technical issues, please email to ccdsgpu-tc@ntu.edu.sg
|                                                                        <#
| \-----------------------------------------------------------------------/
End of banner message from server
scetest6@10.96.189.12's password:
```

Best practice to exist the SSH session: type "**exit**" or press the keys **Ctrl + D**

## SFTP via WinSCP/FileZilla – For file transfer between your local host machine and GPU cluster

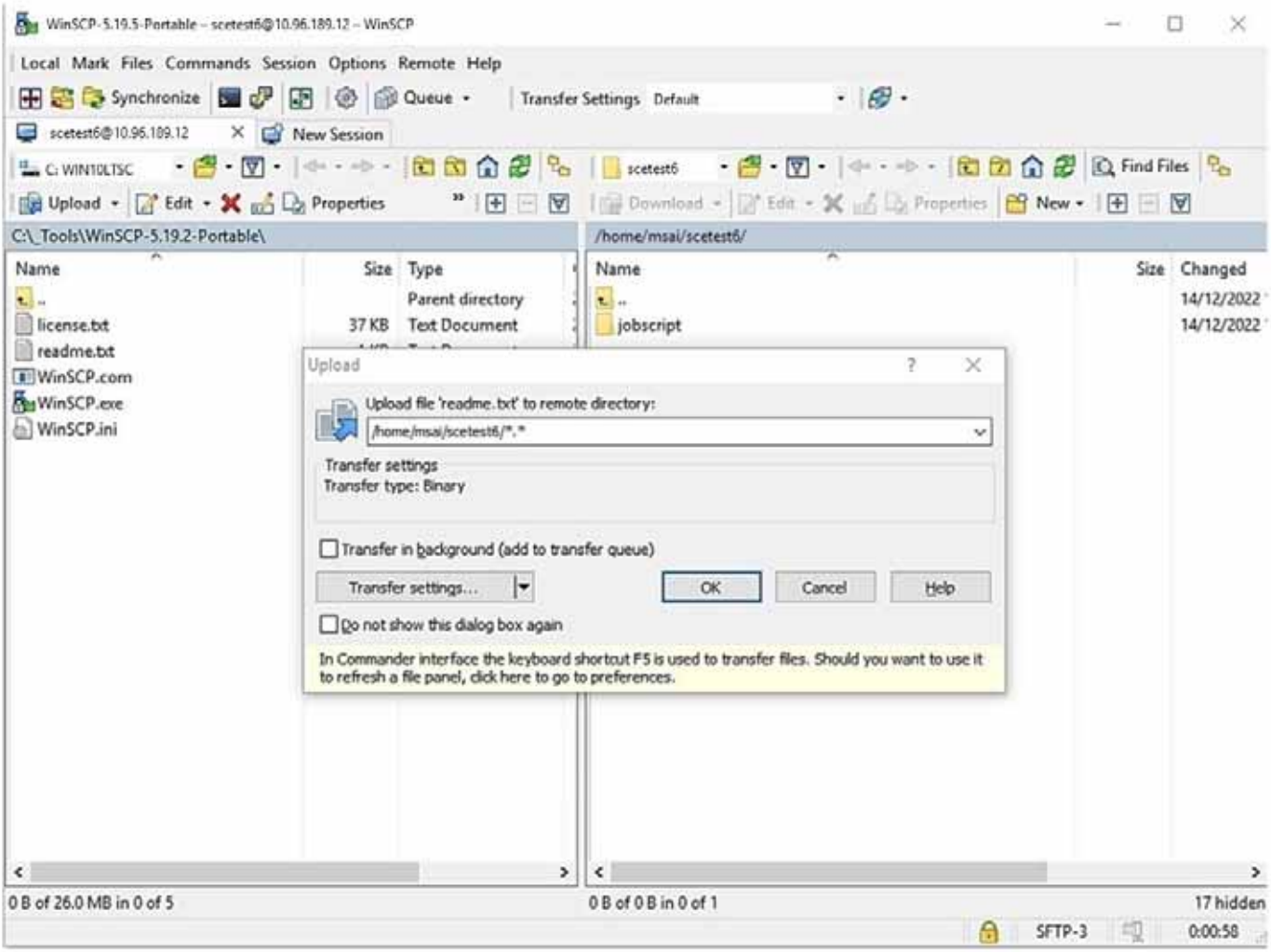
1. In this example, the client tool using for file transfer is WinSCP.
2. Launch WinSCP. Check that the File Protocol is set to SFTP.
3. Under the Host Name, key in the IP Address [10.96.189.12]. Ensure that Port is 22. Key in your NTU Network Account ID (in lower case) and password for the User name and Password field.




4. Ensure that you are in the correct directory before transferring your files over.
   You can verify the pathname of your home directory with following command in SSH session:
   ***>> pwd***

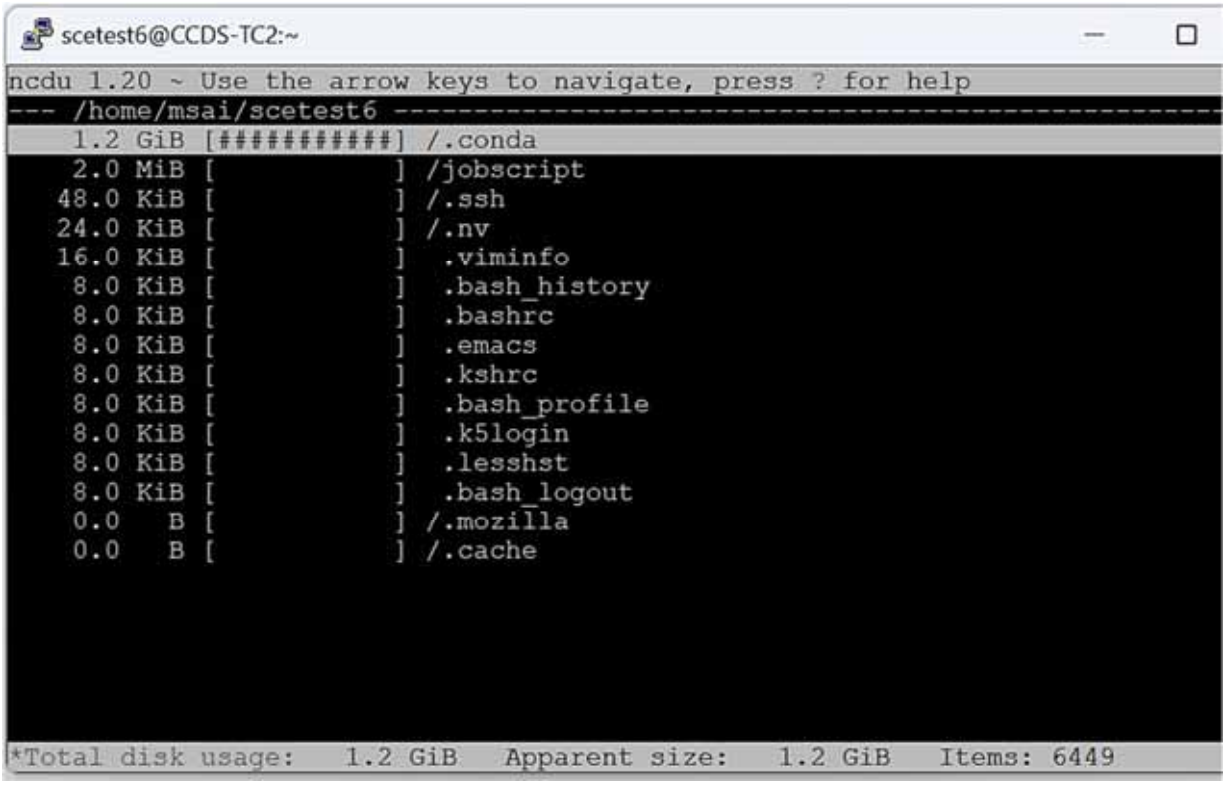
```
[scetest6@CCDS-TC2 ~]$ pwd
/home/msai/scetest6
[scetest6@CCDS-TC2 ~]$
```

Select the file that you want transfer and press the key "*F5*"
The *Upload-Window* will pop out to transfer the file from your host machine to your home directory.
The *Download-Window* will pop out to transfer file from your home directory to selected local disk in your host machine.


WinSCP-5.19.5-Portable – scetest6@10.96.189.12 – WinSCP
Local Mark Files Commands Session Options Remote Help
Synchronize
Queue
Transfer Settings Default
scetest6@10.96.189.12
New Session
C: WIN10LTSC
scetest6
Find Files
Upload
Edit
Properties
Download
New
C:_Tools\WinSCP-5.19.2-Portable\
/home/msai/scetest6/
Name
Size
Type
Parent directory
license.txt
37 KB
Text Document
readme.txt
WinSCP.com
WinSCP.exe
WinSCP.ini
Changed
14/12/2022
jobscript
Upload
Upload file 'readme.txt' to remote directory:
/home/msai/scetest6/*.*
Transfer settings
Transfer type: Binary
Transfer in background (add to transfer queue)
Transfer settings...
OK
Cancel
Help
Do not show this dialog box again
In Commander interface the keyboard shortcut F5 is used to transfer files. Should you want to use it to refresh a file panel, click here to go to preferences.
0 B of 26.0 MB in 0 of 5
0 B of 0 B in 0 of 1
17 hidden
SFTP-3
0:00:58

## Maintain Data in Your Directory

1. Your data and computation are bounded within your assigned home directory and share folder (if any).
2. All home directories are set with disk quota limitation. The student users may have disk quota of 100GB, depending on their usage assignment.
3. The user may encounter following issues when the quota usage reached more than 98%:
   - Unable to login
   - Unable to compute data
   - Unable to save in new data, or having data lost
4. You should do regular housekeeping, ensure there is sufficient free space for your computation.
5. Command to verify the disk usage at command line: *ncdu*




Press the key "**q**" to abort disk scanning or exit.

*Reference on ncdu: https://ostechnix.com/check-disk-space-usage-linux-using-ncdu/*

6. Best practices to ensure enough disk space for your computation:
   - Regularly check on the disk usage for your home directory
   - Transfer and backup your data to your personal device
   - Remove unwanted data in your home directory
   - *Do not remove those system directories and files, which naming with a "." in front, such as ".bash_profile", ".bashrc", ".config" etc*
   - The purpose of the system folder "*.conda*" is to store the packages and environments that you have setup and installed for your computation. *Remove those unwanted environments to free the disk space.*

# Conda Package and Environment

The module package tool is available on the GPU Cluster, allowing users to easily configure their environment based on the application needed. As this cluster is shared among students of different courses in CCDS, there may be some applications that are not relevant to you.

To view the available share applications, apply the modules under */cm/shared/modulefiles*

*>> module avail*

```
[scetest6@CCDS-TC2 ~]$ module avail
------------------------------------------------------- /cm/local/modulefiles -------------------------------------------------------
boost/1.81.0        cm-scale/cm-scale.module  freeipmi/1.6.14  lua/5.4.7     mpc/1.3.1   python3          slurm/slurm/24.11
cluster-tools/11.0  cm-setup/11.0             gcc/14.2.0       luajit        mpfr/4.2.1  python312
cm-bios-tools       cmd                       gdb/16.2         mariadb-libs  null        rocm-smi/4.3.0
cm-image/11.0       cmsh                      gmp/6.3.0        module-git    openldap    sedutil/1.16.0
cm-nvfwupd/2.0.5    dot                       ipmitool/1.8.19  module-info   prs         shared

------------------------------------------------------- /cm/shared/modulefiles ------------------------------------------------------
anaconda/25.5.1     default-environment                hdf5_18/1.8.21        miniconda3/25.5.1           openblas/dynamic/0.3.28
blas/gcc/64/3.11.0  fftw3/openmpi/gcc/64/3.3.10        hwloc/1.11.13         mpich/ge/gcc/64/4.1.1       openmpi/gcc/64/4.1.5
bonnie++/2.00a      gcc/15.1                           hwloc2/2.8.0          mvapich2/gcc/64/2.3.7       openmpi4/gcc/4.1.5
cm-pmix3/3.1.7      gdb/16.3                           iozone/3.494          netcdf/gcc/64/gcc/64/4.9.2  ucx/1.18.0
cm-pmix4/4.1.3      globalarrays/openmpi/gcc/64/5.8    iperf/3.17.1          netperf/2.7.0
cuda/12.8.0         hdf5/1.14.0                        lapack/gcc/64/3.11.0  openblas/dynamic/(default)
[scetest6@CCDS-TC2 ~]$
```

To view the description of a module:

*>> module show <module_name>*

To load the selected module for operation:

*>> module load <module_name>*

To list loaded module:

*>> module list*

To unload the selected module:

*>> module unload <module_name>*

To unload all loaded modules:

*>> module purge*

To know more about the command "module":

*>> module --help*

## Setup Your own Conda Environment

*In each GPU Cluster, the user has no right to execute Sudo and install applications to the system folders*.

You may download and install the Anaconda or Miniconda in your home directory for operation.

**Reference links:**

*https://www.anaconda.com/download/success*

*https://docs.conda.io/en/latest/miniconda.html*

# **There are also available installers under** */apps/_installers* #

```
[scetest6@CCDS-TC2 ~]$ cd /apps/_installers/
[scetest6@CCDS-TC2 _installers]$ ll
total 1242280
-rwx------ 1 root root 1111344533 Jun 24 02:32 Anaconda3-2025.06-0-Linux-x86_64.sh
-rwx------ 1 root root  159476510 Jun 25 02:45 Miniconda3-py313_25.5.1-0-Linux-x86_64.sh
[scetest6@CCDS-TC2 _installers]$
```

OR use the available Anaconda in this GPU Cluster:

To load the module of **anaconda**:

*>> module load anaconda*

To view the information of the loaded **anaconda**:

*>> conda info*

**The Conda commands can only run after the module is loaded.** By default, the read-only base environment is activated. There are ready packages in the centralised Anaconda (or the base environment), you may issue the following command to view the list:

*>> conda list*

You are **not allowed** to install any packages in the base environment. If you find those packages in the base environment do not meet your requirement, you may setup own conda environment in the home directory, and install the require packages for your operation.

For first-time running Conda, the system may prompt you to execute the command "*conda init <shell>*".
The purpose of the command is to write some shell code into the system file (*e.g. ~/.bashrc*) as startup script for conda. For default shell using bash, execute the following

*>> conda init bash*

For the change to take effect, execute

*>> source .bashrc*

## Commands to manage the Conda environment:

| Description | Examples of Command Execution |
|---|---|
| To create a new Conda environment naming as "*TestEnv*" | *>> conda create -n TestEnv* |
| Activate the environment "*TestEnv*" | *>> conda activate TestEnv* |
| To exit the environment "*TestEnv*" | *>> conda deactivate* |
| List the environment created in your home directory | *>> conda env list* |
| To export the configuration of an environment into a *.yml* file. | *>> conda env export > TestEnv.yml* |
| To create the environment from the existing *.yml* file | *>> conda env create -f TestEnv.yml* |
| To update the content of an existing *.yml* file, with option "*prune*" to remove the outdated configuration and dependencies | *>> conda env update --prefix ./TestEnv --file TestEnv.yml --prune* |
| To remove the environment "*TestEnv*" | *>> conda env remove -n TestEnv* |

**Reference on managing the environment:**
*https://docs.conda.io/projects/conda/en/latest/user-guide/tasks/manage-environments.html#creating-an-environment-with-commands*

## Commands to manage the variables in the environment:

| Description | Examples of Command Execution |
|---|---|
| List the variables in the environment | *>> conda env config vars list* |
| To set a new variable in the environment "*my_var1*" | *>> conda env config vars set my_var1=value* |
| To unset (remove) the variable "*my_var1*" | *>> conda env config vars unset my_var1* |
| Always re-activate the environment after adding a new variable and removing a variable | *>> conda activate TestEnv* |

```
[scetest6@CCDS-TC2 ~]$ conda activate TestEnv
(TestEnv) [scetest6@CCDS-TC2 ~]$ conda env config vars list
(TestEnv) [scetest6@CCDS-TC2 ~]$ conda env config vars set my_var1=10
To make your changes take effect please reactivate your environment
(TestEnv) [scetest6@CCDS-TC2 ~]$ conda activate TestEnv
(TestEnv) [scetest6@CCDS-TC2 ~]$ conda env config vars list
my_var1 = 10
(TestEnv) [scetest6@CCDS-TC2 ~]$ conda env config vars unset my_var1
To make your changes take effect please reactivate your environment
(TestEnv) [scetest6@CCDS-TC2 ~]$ conda activate TestEnv
(TestEnv) [scetest6@CCDS-TC2 ~]$ conda env config vars list
(TestEnv) [scetest6@CCDS-TC2 ~]$
```

## Commands to manage the packages in the environment

*The user must always create a new environment to install the package for own use.*

| Description | Examples of Command Execution |
|---|---|
| Search for available packages. For this example, search for available python to install | *>> conda search python*<br>*>> conda search python=3.9* |
| To search and install available package from third-party channel<br><br>The conda package manager usually installs the package from the official default channels. You must specify the channel in the command if to search and install package from third-party channel<br><br>*conda-forge* is one of the third-party channels, providing latest conda packages.<br>**Reference links:**<br>*https://docs.conda.io/projects/conda/en/latest/user-guide/concepts/channels.html*<br>*https://ostechnix.com/enable-conda-forge-channel-for-conda-package-manager/* | # To search for a package in 3rd party channel "conda-forge"<br>*>> conda search -c conda-forge <name of package>*<br><br># To install the package from the selected third-party channel<br>*>> conda install -c conda-forge <name of package>* |
| List the installed packages | *>> conda list* |
| Install selected version of the package | *>> conda install python=3.9.7* |
| The GPU Compute Nodes are ready with their own version of Cuda by NVIDIA, which may not be up-to-date or applicable for user's operation. The user may setup own Conda Environment, and install the desired version of Cuda Toolkit for operation.<br><br>**Reference links:**<br>*https://docs.nvidia.com/deploy/cuda-compatibility/index.html*<br>*https://anaconda.org/nvidia/cudatoolkit* | `[scetest6@CCDS-TC2 ~]$ module avail \| grep cuda`<br>`cuda/12.8.0  mpich/ge/gcc/64/4.1.1`<br><br># To apply Cuda available in the Nodes for **SCSE GPU CLUSTER TC2**, add the module loading command in the job script<br>*>> module load cuda/12.8.0*<br><br># Search for available version of Cuda Toolkit<br>*>> conda search cudatoolkit*<br>*>> conda search -c nvidia cudatoolkit*<br><br># Then install the select version for operation<br>*>> conda install cudatoolkit=<version>*<br>*>> conda install -c nvidia cuda-toolkit=<version>* |
| Remove a package | *>> conda uninstall python=3.9* |

**Reference links on installation of packages:**
*https://www.anaconda.com/docs/tools/working-with-conda/packages/install-packages*
*https://docs.conda.io/projects/conda/en/latest/user-guide/tasks/manage-pkgs.html#installing-packages*

**IMPORTANT NOTE:**
Advise to install the packages using *conda install*. Only use *pip install* if the required package is not available in conda.
This is to prevent package incompatibility.

If you want your conda environment to be loaded each time you start your SSH session, you may append "module load anaconda/25.5.1" and "conda activate yourenvname" at the end of the system file "*.bashrc*", located at your home directory.

```
# .bashrc

# Source global definitions
if [ -f /etc/bashrc ]; then
        . /etc/bashrc
fi

# User specific environment
if ! [[ "$PATH" =~ "$HOME/.local/bin:$HOME/bin:" ]]
then
    PATH="$HOME/.local/bin:$HOME/bin:$PATH"
fi
export PATH

# Uncomment the following line if you don't like systemctl's auto-paging feature:
# export SYSTEMD_PAGER=

# User specific aliases and functions
if [ -d ~/.bashrc.d ]; then
        for rc in ~/.bashrc.d/*; do
                if [ -f "$rc" ]; then
                        . "$rc"
                fi
        done
fi

unset rc

module load anaconda/25.5.1
conda activate TestEnv
```

## Remove unwanted Conda environment, to free disk space

1. Exit from the conda environment >> *conda deactivate*
2. List the conda environment in your home directory >> *conda env list*
3. Only remove unwanted environment in your home directory>> *conda env remove --name <Name of ENV to remove>*

```
[scetest6@CCDS-TC2 ~]$ conda env list

# conda environments:
#
base                  /apps/anaconda3-2025.06-0
TC                    /home/msai/scetest6/.conda/envs/TC
TestEnv               /home/msai/scetest6/.conda/envs/TestEnv
envtest               /home/msai/scetest6/.conda/envs/envtest
mm2lab                /home/msai/scetest6/.conda/envs/mm2lab
mmlab                 /home/msai/scetest6/.conda/envs/mmlab
py38-tfgpu            /home/msai/scetest6/.conda/envs/py38-tfgpu

[scetest6@CCDS-TC2 ~]$ conda env remove --name envtest
2 channel Terms of Service accepted

Remove all packages in environment /home/msai/scetest6/.conda/envs/envtest:
```

# SLURM User Guide

**SLURM** ( *Simple Linux Utility for Resource Management* ) is a software package for submitting, scheduling, and monitoring jobs on large computer clusters.

In each GPU cluster, you are **not allowed** to compile and run your code on the head node. You must create a job script to submit your computation request (**non-interactive jobs**) to SLURM Scheduler, for the system to process in allocated GPU Compute Nodes.

The resources to utilise for GPU computation is limited by the **QoS** (SLURM-Quality of Service) assigned to you.

Command to view the QoS assigned to the user
*>> sacctmgr show user <username> withassoc format=user,qos*

*# You may copy and paste the commands to the command prompt in your SSH session window for execution*
*# Replace the text "<username>" with your own username*

```
[scetest6@CCDS-TC2 ~]$ sacctmgr show user scetest6 withassoc format=user,qos
      User                  QOS
---------- --------------------
  scetest6               normal
[scetest6@CCDS-TC2 ~]$
```

*# The above example shows the user "scetest6" can use the assigned QoS "normal" for GPU computations.*

---

Command to show the resources configured for the QoS:
**>> sacctmgr -P show qos <Name of QoS> withassoc format=name,MaxTRESPU,MaxJobsPU,MaxWall**
**>> sacctmgr show qos <Name of QoS> withassoc format=name%+15,MaxTRESPU%+40,MaxJobsPU%+10,MaxWall%+10**

*# %+<value> is to add value to expand the viewing field*

```
[scetest6@CCDS-TC2 ~]$ sacctmgr -P show qos normal withassoc format=name,MaxTRESPU,MaxJobsPU,MaxWall
Name|MaxTRESPU|MaxJobsPU|MaxWall
normal|cpu=10,gres/gpu=1,mem=30G|2|06:00:00
[scetest6@CCDS-TC2 ~]$ sacctmgr show qos normal withassoc format=name%+15,MaxTRESPU%+40,MaxJobsPU%+10,MaxWall%+1
          Name                                   MaxTRESPU  MaxJobsPU    MaxWall
--------------- ---------------------------------------- ---------- ----------
        normal                 cpu=10,gres/gpu=1,mem=30G          2   06:00:00
[scetest6@CCDS-TC2 ~]$
```

Example showing the Resources assigned to QoS name "**normal**":

1. MaxTRESPU ( *Maximum Trackable RESources Per User* ):
   cpu ( *Maximum number of CPUs/CORES for the user to deploy* ) = 10
   gres/gpu ( *Maximum number of GPU for the user to deploy* ) = 1
   mem ( *Maximum size of Memory for the user to deploy* ) = 30G
2. MaxJobsPU ( *Maximum number of Jobs Per User, can run at a given time* ) = 2
3. MaxWall ( *Maximum Wall clock time per user, to run the job, DD-HH:MM:SS*) = 6-hr

---

The SLURM job script is required, to specify the necessary resources, application, and path to execute your code. Then, submit the job script to SLURM at command line.

**References links on creating the SLURM job script:**
*https://slurm.schedmd.com/quickstart.html*
*https://svante.mit.edu/use_slurm.html*

**Common SLURM commands for you to manage the job submission :**

| Command | Definition (#) & Use Examples (>>) |
|---|---|
| scontrol | *# View the details of a running job, based on its jobid and showing the id of GPU card in the node running the job, under the line "Nodes=TC2N01..."*<br>**>> scontrol show -d jobid <jobid>**<br><br>NumNodes=1 NumCPUs=2 NumTasks=1 CPUs/Task=1 ReqB:S:C:T=0:0:*:*<br>ReqTRES=cpu=1,mem=5G,node=1,billing=1<br>AllocTRES=cpu=2,node=1,billing=2<br>Socks/Node=* NtasksPerN:B:S:C=1:0:*:* CoreSpec=*<br>JOB_GRES=(null)<br>Nodes=TC2N01 CPU_IDs=0-1 Mem=0 GRES=<br>MinCPUsNode=1 MinMemoryNode=5G MinTmpDiskNode=0<br>Features=(null) DelayBoot=00:00:00<br>OverSubscribe=OK Contiguous=0 Licenses=(null) Network=(null)<br><br>*# View the status of the nodes in the Partition "MGPU-TC2"*<br>**>> scontrol show node TC2N0[1-6]**<br><br>NodeName=TC2N03 Arch=x86_64 CoresPerSocket=32<br>CPUAlloc=2 CPUEfctv=128 CPUTot=128 CPULoad=0.00<br>AvailableFeatures=location=local<br>ActiveFeatures=location=local<br>Gres=gpu:nvidia:4(S:0-1)<br>NodeAddr=TC2N03 NodeHostName=TC2N03 Version=24.11.5<br>OS=Linux 5.14.0-503.14.1.el9_5.x86_64 #1 SMP PREEMPT_DYNAMIC Fri Nov 15 12:04:32 UTC 2024<br>RealMemory=463919 AllocMem=0 FreeMem=511644 Sockets=2 Boards=1<br>State=MIXED ThreadsPerCore=2 TmpDisk=0 Weight=1 Owner=N/A MCS_label=N/A<br>Partitions=MGPU-TC2<br>BootTime=2025-08-08T13:18:56 SlurmdStartTime=2025-08-08T15:08:37<br>LastBusyTime=2025-08-08T13:20:03 ResumeAfterTime=None<br>CfgTRES=cpu=128,mem=463919M,billing=128,gres/gpu=4,gres/gpu:nvidia=4<br>AllocTRES=cpu=2,gres/gpu=1,gres/gpu:nvidia=1<br>CurrentWatts=0 AveWatts=0<br><br>*You can reference the resource status of the node, based on the information at the lines "CfgTRES" and "AllocTRES". The values shown under "AllocTRES" are the resources being allocated to the running jobs in the node.* |
| sinfo | *# Display state of the compute nodes in respective partition*<br>**>> sinfo -N -l**<br><br>[scetest6@CCDS-TC2 ~]$ sinfo -N -l<br>Mon Aug 11 10:00:27 2025<br>NODELIST NODES PARTITION STATE CPUS S:C:T MEMORY TMP_DISK WEIGHT AVAIL_FE REASON<br>TC2N01 1 MGPU-TC2* idle 104 2:26:2 347693 0 1 location none<br>TC2N02 1 MGPU-TC2* idle 104 2:26:2 347693 0 1 location none<br>TC2N03 1 MGPU-TC2* mixed 128 2:32:2 463919 0 1 location none<br>TC2N04 1 MGPU-TC2* idle 128 2:32:2 463919 0 1 location none<br>TC2N05 1 MGPU-TC2* idle 128 2:32:2 463919 0 1 location none<br>TC2N06 1 MGPU-TC2* idle 128 2:32:2 463919 0 1 location none<br><br>*For STATE not showing "idle" or "mix", may indicate the node under NODELIST has been down* |
| sacct | *# To view your own job history*<br>**>> sacct --format=jobid,jobname%+15,qos,alloctres%+50,nodelist,start,elapsed,state,reason%+20**<br><br>*# To view all job history for the day, option "-a"*<br>**>> sacct -a --format=user,jobid,jobname%+15,qos,alloctres%+50,nodelist,start,elapsed,state,reason%+20**<br><br>*# Job viewing, excluding those lines with "batch..."*<br>**>> sacct -a --format=user,jobid,jobname%+15,qos,alloctres%+50,nodelist,start,elapsed,state,reason%+20 \| grep -wv batch**<br><br>*# List the jobs executed by specific user <username>*<br>**>> sacct -u <username>** |

| | |
|---|---|
| sbatch | *# Submit a job script to the job queue*<br>**>> sbatch job.sh** |
| scancel | *# The jobid can be obtained by the command "squeue"*<br>*# Cancel a job base on its jobid*<br>**>> scancel <jobid>** |
| squeue | *# Display the jobs in the job queue*<br>**>> squeue**<br><br>*# List more available information with option 'la'*<br>**>> squeue -la**<br><br>*# List the running jobs submitted by the user <username>*<br>**>> squeue -u <username>** |
| seff | *# Slurm Job Efficiency Report*<br>*# Display the statistics of resource being utilized by the completed job, allow to review on the resource assignment for the job*<br>**>> seff <jobid of completed job>** |

# Guideline for SLURM Job Submission

1. **Only apply the QoS assigned to you**
   *The job submitted with unauthorized QoS (QoS not assigned for your use) will be terminated by the system with no prior notification.*

2. **Apply the QoS resources within the limit in your job script**

You may use customized script to view your account and QoS Info

*>> mytcinfo*

```
[scetest6@CCDS-TC2 ~]$ mytcinfo
-------User Info of scetest6 in TC Cluster----------------------
uid=16966075(scetest6) gid=16966075(scetest6) groups=16966075(scetest6),17184922(tc2users)
Home Directory = /home/msai/scetest6
========User Info of scetest6 in SLURM DB=========================================
          User              Cluster                          QOS
--------------- -------------------- ------------------------------
       scetest6                slurm                       normal

===QOS Info=====================================================================
          Name                        MaxTRESPU       MaxJobsPU         MaxWall
--------------- ------------------------------ --------------- ---------------
        normal       cpu=10,gres/gpu=1,mem=30G               2        06:00:00
     Partition
--------------
 MGPU-TC2

-----End---------------------------------------------------------
```

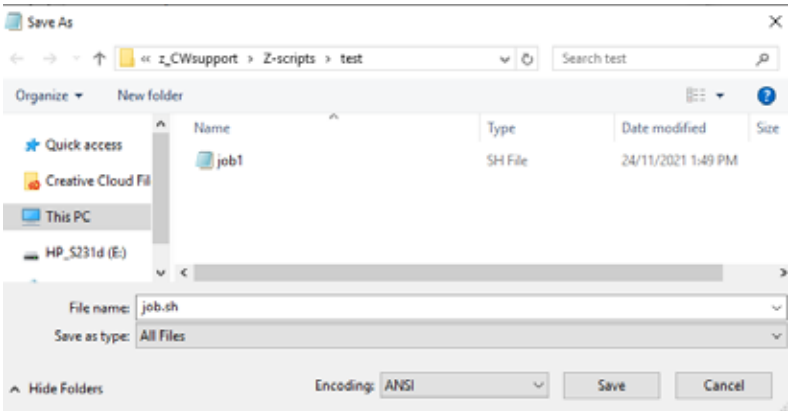
In your job script, ensure to apply the job flag with stated value below or equal to (**<=**) the amount in the assigned QoS.

| | |
|---|---|
| *MaxJobsPU* | The maximum of number of jobs allow to run at a time **<= 2**<br>*You may submit two jobs to run at a time* |
| *MaxWall* | Total computation of your running jobs must not exceed the maximum wall time **<= 6-hr** |
| *MaxTRESPU* | Total amount of resources allocated in your two job scripts must not exceed the maximum values.<br>*For example: No of CPU/cores <=10, Memory <=30GB and No of GPU Card=1, as stated in QoS assigned.* |

*Note: The QoS resource values are subjected to change due to operational requirements*
*# Other customized scripts/commands are available under /tc2share/tc-scripts #*

3. **Create/Edit your SLURM job script**

You may use any text editor available at your host machine, to create the job script, ensuring the job script is saved with file extension "*.sh* ". Then, transfer to your home directory in the GPU Cluster to execute.




In the Head Node, you may use those linux command line editors to edit your job script: *vi*, *nano* or *ne*

**Some references links on Linux Command-line Editors:**
*https://www.guru99.com/the-vi-editor.html*
*https://www.geeksforgeeks.org/nano-text-editor-in-linux/*

**The following are the basic job flags to apply in the job script:**

| Resource | Flag Syntax | Description |
|---|---|---|
| Partition* | --partition=MGPU-TC2 | # Specify the node group for your job execution, defines as *Partition*<br># Command to list the available Partitions and allowed QoS in the Cluster<br>*>> scontrol show partition* |
| QoS* | --qos=normal | # Specify the QoS to apply for the job<br># The QoS must be one of them allow to execute in the Partition |
| nodes* | --nodes=1 | # Number of compute nodes for the job<br># Max node allows per job: 1 |
| | --nodelist=<hostname> | # *Optional*: you may specify the node (hostname) to run the job<br># Command to list the idle nodes and their hostname in the Partition<br>*>> sinfo -N -p <Name of Partition> \| grep idle* |
| GPU* | --gres=gpu:1 | # Specify the use of GPUs on compute nodes.<br># The number of GPU card to use is corresponding to the assigned QoS<br># This option must be present in the job script, for the system to deploy GPU resource |
| Memory* | --mem=1000M<br>Or<br>--mem=1G | # Specify on the memory to apply for the job.<br># The value is corresponding to the assigned QOS, must not exceed the assigned maximum memory size.<br># This option must be present in the job script, for the system to deploy the memory for the computation. |
| CPUs/cores | --ntasks-per-node | # *Optional*: only add this flag if want to apply more than one core for the computation<br># Specify the number of "tasks" (cores) per node, for use with distributed parallelism. *Default value=1* |
| | --cpus-per-task=1 | # *Optional*: only add this flag if want to apply more than one core for the computation<br># Specify the number of CPU-cores to allocate to per task, for use with shared memory parallelism. *Default value=1* |
| Job name* | --job-name=MyJob | # Name of job |
| Output file* | --output=output_%j.out<br>Or<br>--output=output_%x_%j.out | # State the name of the file for standard output<br>*Filename patterns:*<br>*%x: job name*<br>*%j: job id, generated by SLRUM* |
| Error file* | --error=error_%j.err<br>Or<br>--error=error_%x_%j.err | # State the name of the file for error log – if any |
| Time* | --time=01:00:00 | # Specify a time limit for the job – hh:mm:ss<br># The max timing is corresponded to *MaxWall* in assigned QOS |

*The flags with * must specify in the job script*

The following is an example of a standard job script: job.sh

```
#!/bin/bash
#SBATCH --partition= MGPU-TC2
#SBATCH --qos=normal
#SBATCH --nodes=1
#SBATCH --gres=gpu:1
#SBATCH --time=01:00:00
#SBATCH --mem=1G
#SBATCH --job-name=MyJob
#SBATCH --output=output_%x_%j.out
#SBATCH --error=error_%x_%j.err


module load anaconda
eval "$(conda shell.bash hook)"
conda activate TestEnv
```

*Remarks:*
*You may add necessary flags from the table above. Do note that if you exceed the limit for your QOS, the job will not run.*
*From this job script, 2 files will be created:*
*output_%x_%j.out*
- *The standard output from running the code will be saved here*

*error_%x_%j.err*
- *Error log from job if any*

*Load necessary modules needed to run the code*
*While trying to activate your conda environment, you may encounter a CommandNotFoundError.*

```
CommandNotFoundError: Your shell has not been properly configured to use 'conda activate'.
To initialize your shell, run

    $ conda init <SHELL_NAME>

Currently supported shells are:
  - bash
  - fish
  - tcsh
  - xonsh
  - zsh
  - powershell

See 'conda init --help' for more information and options.

IMPORTANT: You may need to close and restart your shell after running 'conda init'.
```

*Initialize the conda environment using* eval "$(conda shell.bash hook)".
*Activate your conda environment (if any)*
*Run your code*

## 4. Submit your job script

| Action to submit a job | Command |
|---|---|
| To submit your job script, where job.sh is the name of your job script | **sbatch job.sh** |

The system should respond with a job ID:

```
[scetest6@CCDS-TC2 sample-1]$ sbatch job.sh
Submitted batch job 62
```

## 5. Avoid using the command "*srun*" to submit job

The command "*srun*" is to submit job at the command line for *real time execution*. You have to maintain your SSH session until the whole process completed. The disconnection of your SSH session may kill the process and cause you to lose the control over the execution.

The command "*sbatch*" is to submit job for later execution, handling by SLURM in the background.
Once submitted the job using "*sbatch*", you may exit from your SSH session, with no consequence.

To avoid high volume of SSH connections to the Cluster Head Node, all users are advised to use the command "*sbatch*" for job submission.

## 6. Verify on the job queue and node operational status

Command to verify on the job queue:
*>> squeue*

Command to verify on the resources being in used by the running jobs
*For nodes under partition "MGPU-TC2" >> scontrol show node TC2N0[1-6]*

*# Look at the overall resources being allocated for node operation and running jobs*

```
NodeName=TC2N03 Arch=x86_64 CoresPerSocket=32
   CPUAlloc=2 CPUEfctv=128 CPUTot=128 CPULoad=0.00
   AvailableFeatures=location=local
   ActiveFeatures=location=local
   Gres=gpu:nvidia:4(S:0-1)
   NodeAddr=TC2N03 NodeHostName=TC2N03 Version=24.11.5
   OS=Linux 5.14.0-503.14.1.el9_5.x86_64 #1 SMP PREEMPT_DYNAMIC Fri Nov 15 12:04:32 UTC 2024
   RealMemory=463919 AllocMem=0 FreeMem=511644 Sockets=2 Boards=1
   State=MIXED ThreadsPerCore=2 TmpDisk=0 Weight=1 Owner=N/A MCS_label=N/A
   Partitions=MGPU-TC2
   BootTime=2025-08-08T13:18:56 SlurmdStartTime=2025-08-08T15:08:37
   LastBusyTime=2025-08-08T13:20:03 ResumeAfterTime=None
   CfgTRES=cpu=128,mem=463919M,billing=128,gres/gpu=4,gres/gpu:nvidia=4
   AllocTRES=cpu=2,gres/gpu=1,gres/gpu:nvidia=1
   CurrentWatts=0 AveWatts=0
```

- The node selected to run the job is based on the available resources (CPU, Memory, GPU and Compute nodes) at the time of submission
- For the jobs failed the assignment will go into PENDING queue
- The running jobs terminated due to node reset or failure, will also be appearing in the PENDING queue

*# How to spot failing node #*

- *For node with state showing "DRAIN", indicating it has failed task. This node is alive but may not able to accept new job and the performance of existing running jobs may also be affected.*
- *Requiring the administrator to reset the affected node for operation. The impact of the reset is the existing running jobs in the affected node will be terminated. The affected jobs will have to join the PENDING queue for re-assignment.*

```
NodeName=TC2N03 Arch=x86_64 CoresPerSocket=16
   CPUAlloc=18 CPUTot=64 CPULoad=20.63
   AvailableFeatures=(null)
   ActiveFeatures=(null)
   Gres=gpu:8(S:0-1)
   NodeAddr=TC2N03 NodeHostName=TC2N03 Version=23.02.7
   OS=Linux 5.14.0-362.8.1.el9_3.x86_64 #1 SMP PREEMPT_DYNAMIC Wed Nov 8 17:36:32 UTC 2023
   RealMemory=128508 AllocMem=89344 FreeMem=84255 Sockets=2 Boards=1
   State=MIXED+DRAIN ThreadsPerCore=2 TmpDisk=0 Weight=1 Owner=N/A MCS_label=N/A
   Partitions=M2
   BootTime=2024-08-11T19:28:31 SlurmdStartTime=2024-08-11T22:41:30
   CfgTRES=cpu=64,mem=128508M,billing=64,gres/gpu=8
   AllocTRES=cpu=18,mem=89344M,gres/gpu=6
   CapWatts=n/a
   CurrentWatts=0 AveWatts=0
   ExtSensorsJoules=n/s ExtSensorsWatts=0 ExtSensorsTemp=n/s
   Reason=Kill task failed [root@2024-08-11T19:45]
```

- Common reasons showing for such affected jobs in the PENDING queue: *Priority*, *Resources*
- The job will be executed when there is ready node and resource to assign within Time Limit (referenced to the maximum wall time *(MaxWall)* stated in the QoS)
- The pending job will be removed by the system when exceeded the Time Limit

Advice:

- If there are many jobs in the PENDING queue with reasons of *"Priority" or "Resources", please d*efer your job submission until those jobs are cleared in the queue

## 7. Process your submitted SLURM job

| Actions to check on job status | Command |
|---|---|
| Display the jobs in the scheduling queue | **squeue -la** |
| Display job history for a user <username> | **sacct -u <username>** |
| Show the detail for a running/pending job <jobid> | **scontrol show jobid <jobid>** |

If the Status indicates PD (PENDING) and NODELIST (REASON) indicates a reason in bracket, do note that the job has failed to run. Please cancel the job if this happens.

```
[scetest6@CCDS-TC2 ~]$ squeue -la
Mon Aug 11 13:48:45 2025
             JOBID PARTITION     NAME     USER    STATE       TIME TIME_LIMI  NODES NODELIST(REASON)
                59  MGPU-TC2  TestJob scetest6  PENDING       0:00   1:00:00      1 (QOSMaxCpuPerUserLimit)
```

For submitted job failed the QoS limit check (*exceeded any of the values stated for the QoS*), will go into the PENDING queue

Common reasons for job associates to QoS failure:

- *QOSMaxJobsPerUserLimit*
  - Verify on your total number of jobs in Running queue, must not exceed the value stated in "**MaxJobsPU**"
  - Extra job will go into the PENDING queue
- *QOSMaxMemoryPerUser, QOSMaxCpuPerUserLimit, QOSMaxGRESPerUser*
  - The total resource count [ **#SBATCH --mem=x, #SBATCH --ntasks-per-node=x** ] stated in all your submitted job scripts using the same QoS, must not exceed the max value stated in "**MaxTRESPU**"
- *QOSMaxGRESPerUser*
  - This error may appear if unable to assign your jobs using the same QoS, to the allocated (1) GPU Card
  - For this case, restrict to submit only one (1) job to run at a time

The job will only be executed when passed the checks for QoS limit and node assignment within the Time Limit (referenced to **MaxWall** stated in the QoS)

The pending job will be removed by the system when reached the Time Limit.

Advice for such encounter:

- Cancel your pending jobs which failed the QoS limit Check
- Review and modify the resource values in your job script
- Restrict your number of job submission

| Action to cancel the job | Command |
|---|---|
| To cancel a job, where 263 is your job id | **scancel 263** |

Once the job has finished running, it will be removed from the queue. You should be able to see the output file. If you did not specify a specific path for the output file in your SLURM script, it will be created in the directory where you submitted the SLURM job.

```
[scetest6@CCDS-TC2 sample-1]$ ls
error_MyJob_91.err  job.sh  output_MyJob_91.out  test.py
[scetest6@CCDS-TC2 sample-1]$
```

## 8. To know more about your completed jobs

You may use the customised script to view your job history for the day, *with details on applied QoS, allocated resources (cpu, gpu, memory assigned for the job), node assigned, job-submit time, start time, elapsed time, state and reason*

*>> myjobhistory*

```
[scetest6@CCDS-TC2 ~]$ myjobhistory
--List Running Jobs--------------------------------------------------
| User | jobid | jobname | qos | allocResources | start | elapsed | nodelist | state | reason
  scetest6     48     TestJob     normal billing=2,cpu=2,gres/gpu:nvidia=1,gres/gpu=1,node+     2025-08-11T10:52:46   00:00:03  TC2N01  COMPLETED  None
  scetest6     49     TestJob     normal billing=2,cpu=2,gres/gpu:nvidia=1,gres/gpu=1,node+     2025-08-11T10:54:46   00:00:03  TC2N01  COMPLETED  None
```

*# Know the actual GPU Card being assigned for your Running job #*

*Using the command "scontrol", with option "-d"*

*>> scontrol -d show jobid <jobid of running job>*

*Look for the line "JOB_GRES=gpu:nvidia:x "*

```
  NodeList=TC2N01
  BatchHost=TC2N01
  NumNodes=1 NumCPUs=2 NumTasks=1 CPUs/Task=1 ReqB:S:C:T=0:0:*:*
  ReqTRES=cpu=1,mem=1G,node=1,billing=1,gres/gpu=1
  AllocTRES=cpu=2,node=1,billing=2,gres/gpu=1,gres/gpu:nvidia=1
  Socks/Node=* NtasksPerN:B:S:C=0:0:*:* CoreSpec=*
  JOB_GRES=gpu:nvidia:1
    Nodes=TC2N01 CPU_IDs=0-1 Mem=0 GRES=gpu:nvidia:1(IDX:0)
  MinCPUsNode=1 MinMemoryNode=1G MinTmpDiskNode=0
  Features=(null) DelayBoot=00:00:00
  OverSubscribe=OK Contiguous=0 Licenses=(null) Network=(null)
```

# In TC2 GPU Cluster, every compute node has GPU Cards which range in number from 4 to 8. The GPU cards are identified by SLURM as *IDX:0, IDX:1, IDX:2, IDX:3, IDX:4, IDX:5, IDX:6 and IDX:7, respectively.*

# The above example showing the job is executing on GPU Card 1 in the node TC2N01

# To know about the actual utilization of the GPU Card processing your job, you may view those history files "*MSAI-ActiveNode-YYYY-MM-DD-HH-MM* ", updating hourly for past 24-hours under */tc2share/Cluster-Information*

# Use the command "*more*" to view those files for the day >> *more MSAI-ActiveNode-2023-04-21-**

*# Know the utilization of the GPU Card assigned to your Running job #*

```
=[ GPU Info @ 2023-04-21-13-00 ]==============================================
TC2N01: name, pci.bus_id, driver_version, pstate, power.limit [W], power.draw [W], temperature.gpu, memory.total
TC2N01: NVIDIA A40, 00000000:12:00.0, 520.61.05, P0, 300.00 W, 156.79 W, 42, 46068 MiB, 6664 MiB, 0 %, 0 %
TC2N01: NVIDIA A40, 00000000:13:00.0, 520.61.05, P0, 300.00 W, 170.60 W, 48, 46068 MiB, 9320 MiB, 48 %, 41 %
TC2N01: NVIDIA A40, 00000000:48:00.0, 520.61.05, P0, 300.00 W, 264.48 W, 65, 46068 MiB, 5448 MiB, 90 %, 73 %
TC2N01: NVIDIA A40, 00000000:49:00.0, 520.61.05, P0, 300.00 W, 221.54 W, 56, 46068 MiB, 7272 MiB, 99 %, 57 %
TC2N01: NVIDIA A40, 00000000:89:00.0, 520.61.05, P0, 300.00 W, 251.43 W, 65, 46068 MiB, 5404 MiB, 88 %, 61 %
TC2N01: NVIDIA A40, 00000000:8A:00.0, 520.61.05, P0, 300.00 W, 72.76 W, 37, 46068 MiB, 6642 MiB, 0 %, 0 %
TC2N01: NVIDIA A40, 00000000:C1:00.0, 520.61.05, P0, 300.00 W, 123.38 W, 42, 46068 MiB, 9856 MiB, 0 %, 0 %
TC2N01: NVIDIA A40, 00000000:C2:00.0, 520.61.05, P0, 300.00 W, 278.60 W, 66, 46068 MiB, 17274 MiB, 100 %, 82 %
```

*The information is retrieved using the command "nvidia-smi" from respective GPU node.*

| Queries Option | Description |
|---|---|
| name | Official Product Name of the GPU Card |
| pci.bus_id | PCI Bus ID |
| driver_version | The version of the installed NVIDIA Display Driver |
| pstate | Current Performance state of the GPU Card, ranging from P0 (maximum performance) to P12 (minimum performance) |
| temperature.gpu | Core GPU Temperature, in Degrees C |
| memory.total [MiB] | Total installed GPU Memory |
| memory.used [MiB] | Total Memory being allocated by the active contexts |
| *utilization.gpu [%] | Percent of time over the past second during which one or more kernels was executing on the GPU |
| *utilization.memory [%] | Percent of time over the past second during which global (device) memory was being read or written |

** Utilization rates indicate how much of the GPU, the assigned job has been using.*
*You are advised to check on the GPU usage and status for your running job, to know whether your coding has been executing correctly and utilizing the resource efficiently.*

*# Know the CPU and Memory being utilized for your Completed Job #*

*Command to display the resource utilization statistics for your completed job, let you review and deploy the right number of resources (CPU/cores and Memory) for your computation.*

**>> seff <job id>**

```
[scetest6@CCDS-TC2 ~]$ seff 83
Job ID: 83
Cluster: slurm
User/Group: scetest6/scetest6
State: COMPLETED (exit code 0)
Cores: 1
CPU Utilized: 00:00:01
CPU Efficiency: 50.00% of 00:00:02 core-walltime
Job Wall-clock time: 00:00:02
Memory Utilized: 1.24 MB
Memory Efficiency: 0.12% of 1.00 GB
```

*# The user may review the resources utilized and adjust the values of the CPU and Memory in the job script accordingly to enhance the performance of subsequent computations*

# Miscellaneous

## SSH Tunneling

The GPU Clusters are only accessible via SSH connection. For the user requiring running *Jupyter Notebooks*, must learn to setup the SSH tunnel for computation.

SSH tunneling (also known as SSH port forwarding) is a method of creating an encrypted SSH connection between a client and a server through which services port can be relayed.

The following is an exemplary guide:
*Running Jupyter Notebook on a computer node that needs to be contacted by the web browser on your local computer*

1. **SSH to GPU Head Node and setup Jupyter Notebook in your home directory**

| Description | Execution |
|---|---|
| Load the anaconda module | *# Execute Command*<br>**>> module load anaconda** |
| Create and activate a conda environment | *# Execute Command*<br>**>> conda create -n RunJupyter**<br>**>> conda activate RunJupyter** |
| Install the required package for Jupyter Notebook from third-party channel "*conda-forge*" | *# Execute Command*<br>**>> conda install -c conda-forge jupyter** |
| Deactivate the conda environment | *# Execute Command*<br>**>> conda deactivate** |

2. **Create the job script**

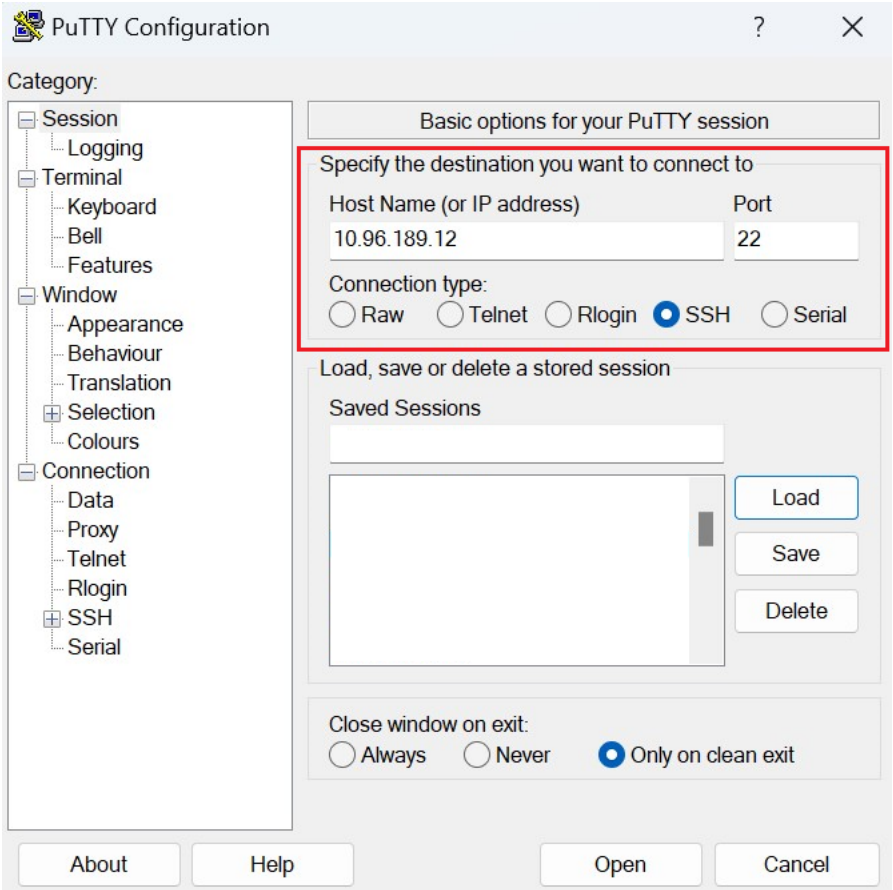
| Description | Execution |
|---|---|
| Create the job script to run Jupyter Notebook. For this example, the job script is named as "*run1.sh*" | *# Sample of the job script*<br><br>*#!/bin/sh*<br>*#SBATCH --partition=MGPU-TC2*<br>*#SBATCH --qos=normal*<br>*#SBATCH --gres=gpu:1*<br>*#SBATCH --nodes=1*<br>*#SBATCH --mem=1G*<br>*#SBATCH --job-name=run1*<br>*#SBATCH --time=01:00:00*<br>*#SBATCH --output=output_%x_%j.out*<br>*#SBATCH --error=error_%x_%j.err*<br><br>*module load anaconda*<br>*eval "$(conda shell.bash hook)"*<br>*conda activate RunJupyter*<br>*jupyter-notebook --ip=$(hostname -i) --port=8887* |
| Submit the job script | *# Execute Command*<br>**>> sbatch run1.sh** |

3. **Verify the access information in the error log.**
   Take note of the *IP address*, *port number* and the *URL* for web access

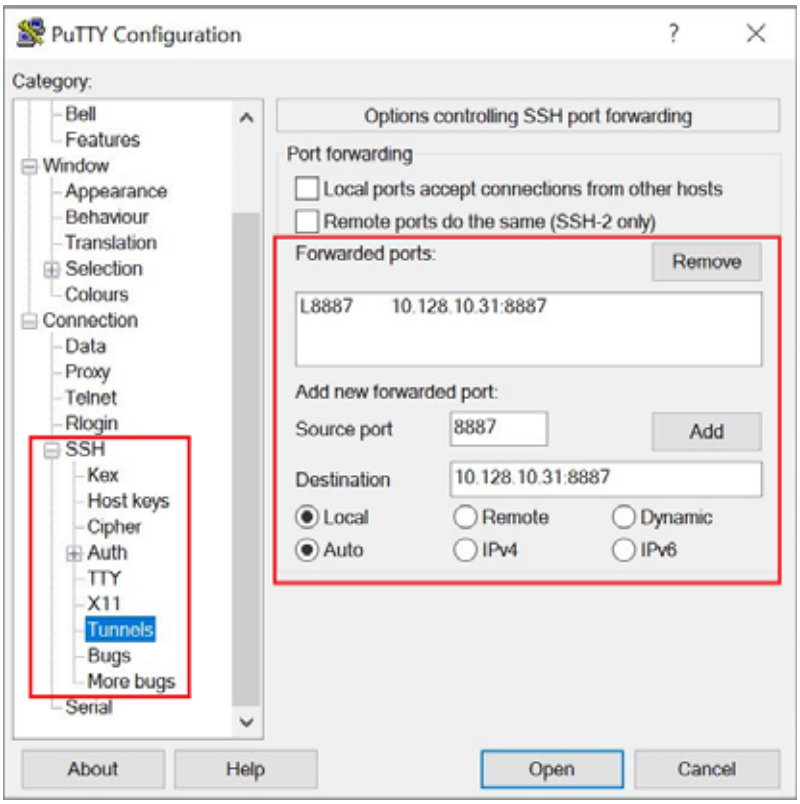
```
[scetest6@CCDS-TC2 jobscript]$ ll
total 24
-rw-r--r-- 1 scetest6 scetest6 2834 Aug 13 10:38 error_run1_109.err
-rw-r--r-- 1 scetest6 scetest6  288 Aug 13 10:38 output_run1_109.out
-rw-r--r-- 1 scetest6 scetest6  357 Aug 13 10:38 run1.sh
[scetest6@CCDS-TC2 jobscript]$ cat error_run1_109.err
[I 2024-08-13 10:38:35.806 ServerApp] jupyter_lsp | extension was successfully linked.
[I 2024-08-13 10:38:35.811 ServerApp] jupyter_server_terminals | extension was successfully linked.
[I 2024-08-13 10:38:35.817 ServerApp] jupyterlab | extension was successfully linked.
[I 2024-08-13 10:38:35.822 ServerApp] notebook | extension was successfully linked.
[I 2024-08-13 10:38:35.827 ServerApp] Writing Jupyter server cookie secret to /home/msai/scetest6/.local/share/jupyter/runtime/jupyter_cookie_secret
[I 2024-08-13 10:38:36.317 ServerApp] notebook_shim | extension was successfully linked.
[I 2024-08-13 10:38:36.356 ServerApp] notebook_shim | extension was successfully loaded.
[I 2024-08-13 10:38:36.358 ServerApp] jupyter_lsp | extension was successfully loaded.
[I 2024-08-13 10:38:36.359 ServerApp] jupyter_server_terminals | extension was successfully loaded.
[I 2024-08-13 10:38:36.370 LabApp] JupyterLab extension loaded from /home/msai/scetest6/.conda/envs/RunJupyter/lib/python3.12/site-packages/jupyterlab
[I 2024-08-13 10:38:36.370 LabApp] JupyterLab application directory is /home/msai/scetest6/.conda/envs/RunJupyter/share/jupyter/lab
[I 2024-08-13 10:38:36.370 LabApp] Extension Manager is 'pypi'.
[I 2024-08-13 10:38:36.384 ServerApp] jupyterlab | extension was successfully loaded.
[I 2024-08-13 10:38:36.390 ServerApp] notebook | extension was successfully loaded.
[I 2024-08-13 10:38:36.390 ServerApp] Serving notebooks from local directory: /home/msai/scetest6/jobscript
[I 2024-08-13 10:38:36.390 ServerApp] Jupyter Server 2.14.2 is running at:
[I 2024-08-13 10:38:36.390 ServerApp] http://10.128.10.31:8887/tree?token=d418e8d53697fa26e8f7cedecac293fcda34a242e0691f36
[I 2024-08-13 10:38:36.390 ServerApp]     http://127.0.0.1:8887/tree?token=d418e8d53697fa26e8f7cedecac293fcda34a242e0691f36
[I 2024-08-13 10:38:36.390 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 2024-08-13 10:38:36.402 ServerApp]

    To access the server, open this file in a browser:
        file:///home/msai/scetest6/.local/share/jupyter/runtime/jpserver-116104-open.html
    Or copy and paste one of these URLs:
        http://10.128.10.31:8887/tree?token=d418e8d53697fa26e8f7cedecac293fcda34a242e0691f36
        http://127.0.0.1:8887/tree?token=d418e8d53697fa26e8f7cedecac293fcda34a242e0691f36
[I 2024-08-13 10:38:36.440 ServerApp] Skipped non-installed server(s): bash-language-server, dockerfile-language-server-nodejs, javascript-typescript-langserver, jedi-langu
age-server, julia-language-server, pyright, python-language-server, python-lsp-server, r-languageserver, sql-language-server, texlab, typescript-language-server, unified-la
nguage-server, vscode-css-languageserver-bin, vscode-html-languageserver-bin, vscode-json-languageserver-bin, yaml-language-server
```

4. **In your local machine, start another SSH session to the cluster which you have accessed:**



5. **Expand the category of "*SSH*" and select "*Tunnels*"**



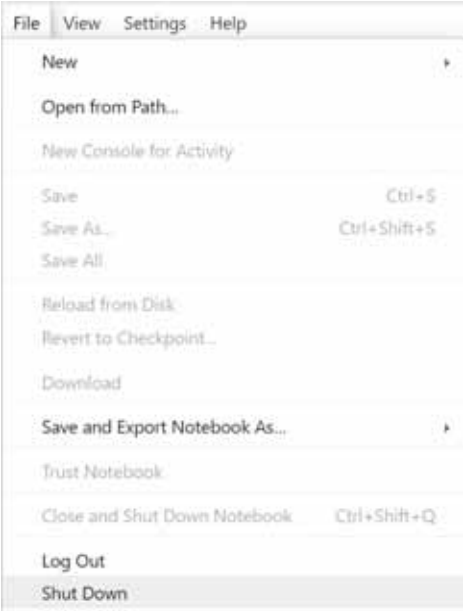

*Input the IP address and port number obtained in the Error Log as show above.*
*Click on the button "Add" to add into the Forwarded ports*

6. **Click on the button "*Open*" to start the SSH session. Login using your NTU username and password to establish the connection.**

7. Finally, you **may access the URL in your local machine**:

```
To access the server, open this file in a browser:
    file:///home/msai/scetest6/.local/share/jupyter/runtime/jpserver-572409-open.html
Or copy and paste one of these URLs:
    http://10.128.10.31:8887/tree?token=4a18d2aab1d63072b36ac516bdfeca5b7c938a1d99dce840
    http://127.0.0.1:8887/tree?token=4a18d2aab1d63072b36ac516bdfeca5b7c938a1d99dce840
```

8. The operation of Jupyter will terminate when you select "*Shut Down*" in the web page. Then, you may logout from the SSH session set with the forwarded port.



# Other Resources for Application

## QoS

The user may apply additional QoS for…

- longer computation hour, selecting range: 8-hr, 12-hr, 24-hr, till max 48-hr
- computation with more resource allocation: CPUs, Memory and GPU Card

Usage period: not more than 1-month

Method to apply:

- Send the request e-mail to CCDS GPU-TC [*ccdsgpu-tc@ntu.edu.sg*]
- Must provide **valid reason** for additional usage requirements and **data** showing that the latest coding and computational results executed on the assigned QoS is unable to support the user's current computational requirements
- The administrator will evaluate, verify on the real requirements, and then assign the QoS accordingly
- The requestor will be notified, upon successful application

## Additional Storage Space

The user may apply additional storage space for computation data more than 100GB
A share folder will be created in another storage to assign to the user.

Usage period: 4-months
Maximum usage quota: depending on the available free space and approval

Method to apply:

- Send the request e-mail to CCDS GPU-TC [*ccdsgpu-tc@ntu.edu.sg*]
- Must provide the reason and data to support the usage request
- The user will be receiving the PDF form, via e-mail, to complete for the application
- The user will be notified of the resource assignment, upon successful application

## Terms and Conditions

- The application is subjected to resource availability, cluster operational workload and administration approval
- The usage of the assigned resource will be monitored
- The administrator claims the right to revoke the assigned resource if the usage has been verified to be underused or impacts the operations of the GPU Cluster
- The assigned resource will be removed upon usage expiry, with no prior notification

# Important Notice

1. System Administrators claim the right to remove the accounts of users and the folders assigned to users without prior notice.

2. **NO backups** will be provided for data stored in folders assigned to users. Users are responsible for protecting and maintaining their own data.

3. Additional user folders and additional QoS assigned to users will be removed after the usage expiry date, without prior notice.

4. Resources are to be used for academic purposes only. Use of resources for purposes other than that for which they were granted will result in the termination of the account.

5. Your computation is being monitored. For jobs submitted with **unauthorised** QOS (**NOT** assigned for your use) and operations, will be terminated with no prior notification. Severe offenders may be barred from the service.

6. **DO NOT** access directories other than your Home Directory or Shared folder. Unauthorised access to other directories or files, even for the purpose of "browsing", shall be deemed as a security breach. Serious action will be taken against users committing any breach to the CCDS GPU Cluster.

7. For issues regarding your coursework and project, please consult your Supervisor or TA (Teaching Assistant)

8. For technical issues, send an email to CCDS GPU-TC [*ccdsgpu-tc@ntu.edu.sg*]

~ End ~